前言：本文围绕自定义多任务感知模型（bevformer+det/maptrv2）的真值生成链路，系统说明 MapTRv2 风格地图 GT 是如何从 nuScenes 中间格式、地图 JSON 和位姿信息中在线生成的。文章重点回答三个问题：

1. 生成 MapTRv2 的 GT 需要哪些数据；
2. 这些数据在生成时分别怎么用；
3. 对应到当前工程，代码分别落在哪些文件里。

目录

- 总述
- 1.当前配置到底在做什么
- 2.需要哪些数据
- 3.完整真值生成流程
- 4.位姿信息的具体作用
- 5.地图 JSON 的实际组织方式
- 6.中心线拓扑是怎么做出来的
- 7.训练时最终写进 batch 的 GT 长什么样
- 8.常见问题

总述

本文面向在线生成MapTRv2 模型需要的监督数据构建，即在给定样本位姿和 `map_location` 的前提下，如何从静态地图 JSON 中裁取局部地图，并将其中的几何结构转成监督信号。其核心流程可以概括为四步：

- 离线整理 nuScenes 原始样本，生成 `nuscenes_infos_temporal_{train,val}.pkl` 等中间索引文件；
- 训练时根据 `map_location` 和位姿信息，从 `data/nuscenes/maps/expansion/<location>.json` 中裁取局部地图 patch；
- 将 patch 内的线、面、拓扑结构转成固定点数的向量监督，得到 `gt_map_vecs_label` 和 `gt_map_vecs_pts_loc`；
- 由 map head 在 one-to-one / one-to-many 查询框架下完成匹配、回归和损失计算。

---

1. 配置说明

当前配置 projects/configs/bevformer/bev_tiny_det_mapv2.py，在 base 配置 projects/configs/bevformer/bev_tiny_det_map_apollo.py 的maptr基础上新增了含有拓扑信息的centerline类别，新增了seg损失，修改了decoder等，对齐Maptrv2核心升级点。

概括而言，官方 MapTRv2 与当前配置的差异主要体现在以下几项：

- MapTRv2 的常见 nuScenes 配置是 `ResNet50 + FPN + LSSTransform`；
- 当前 base 配置采用Apollo-Vision-Net典型结构：`DLA + SECONDFPNV2 + BEVFormerEncoder`；

- 类别配置对齐Maptrv2，新增 `centerline`；
- 对齐MapTRv2 采用 `one-to-one + one-to-many` 的 query 组织，增加 `map_aux_seg` 辅助分割用于模型监督训练。

---

2. 需要哪些数据

MapTRv2 的 GT 生成，真正依赖的是三类数据：

2.1 nuScenes 原始地图文件

目录结构：

```
/home/nuvo/Apollo-Vision-Net/data/nuscenes/maps/expansion/
├── boston-seaport.json
├── singapore-hollandvillage.json
├── singapore-onenorth.json
├── singapore-queenstown.json
```

这里的 JSON 是 nuScenes map expansion 格式，文件后缀是 `.json`。

当前工程通过以下方式读取它们：

- `NuScenesMap(dataroot=self.data_root, map_name=location)`
- 其中 `self.data_root` 通常是 `data/nuscenes`
- `location` 则来自 infos 里的 `map_location`

这部分代码落在：

- projects/mmdet3d_plugin/datasets/nuscenes_det_occ_map_dataset.py

2.2 nuScenes 中间格式 infos

PROF

目录：

```
/home/nuvo/Apollo-Vision-Net/data/nuscenes/
├── nuscenes_infos_temporal_train.pkl
├── nuscenes_infos_temporal_val.pkl
└── nuscenes_infos_temporal_test.pkl   # 如果你生成 test split
```

这些 pkl 是中间格式，不是地图真值本身，而是“每个 sample 的元信息索引表”。

MapTRv2 生成 GT 时主要使用的字段包括：

- `token`
- `scene_token`
- `scene_name`
- `map_location`

- `lidar2ego_translation`
- `lidar2ego_rotation`
- `ego2global_translation`
- `ego2global_rotation`
- `can_bus`
- `cams`
- `sweeps`

这些字段由离线 converter 生成，入口在：

- tools/create_data.py
- tools/data_converter/nuscenes_converter.py

2.3 当前样本的时序图像与标定信息

训练时数据集还会读到：

- 6 相机图像
- lidar top 传感器位姿
- ego pose
- can bus
- 历史 sweep

这些信息在 `prepare_train_data(...)` 里被整理成样本，然后再注入 map GT。

---

3. 完整真值生成流程

这一节按代码执行顺序讲。

3.1 第一步：离线生成 infos

入口命令：

PROF

```
python tools/create_data.py nuscenes \
  --root-path ./data/nuscenes \
  --out-dir ./data/nuscenes \
  --extra-tag nuscenes \
  --version v1.0 \
  --canbus ./data
```

该命令最终会调用：

- `tools/data_converter/nuscenes_converter.py::create_nuscenes_infos(...)`

这里会把每个 sample 的原始信息整理成一条 info，写入：

- `data/nuscenes/nuscenes_infos_temporal_train.pkl`
- `data/nuscenes/nuscenes_infos_temporal_val.pkl`

其中最重要的是：

- `map_location`
  - 通过 `scene_token -> scene -> log -> location` 得到。
- `lidar2ego_*` / `ego2global_*`
  - 位姿信息。
- `can_bus`
  - 由 nuScenes can bus 数据读取。

3.2 第二步：数据集读取 infos，准备训练样本

数据集主入口是：

- projects/mmdet3d_plugin/datasets/nuscenes_det_occ_map_dataset.py

训练时会进入：

- `prepare_train_data(index)`

其流程是：

1. 从 `self.data_infos[index]` 读出 info；
2. 补齐 `scene_name`、`map_location`、`lidar2ego_*`；
3. 执行原有 pipeline；
4. 在 pipeline 后调用 `_add_vectormap_gt(...)`；
5. 把地图真值写回 sample。

关键注入字段：

- `gt_map_vecs_label`
- `gt_map_vecs_pts_loc`

3.3 第三步：根据位姿选地图并裁剪 patch

在 `_add_vectormap_gt(...)` 中会先做：

- 解析当前 sample 的地图位置 `location`
- 读取位姿：
  - `lidar2ego_rotation`
  - `lidar2ego_translation`
  - `ego2global_rotation`
  - `ego2global_translation`
- 组合出 `lidar2global`

然后调用：

```
self.vector_map.gen_vectorized_samples(location,
lidar2global_translation, lidar2global_rotation)
```

这里的 `vector_map` 在 V2 中是：

- `VectorizedLocalMapV2`

代码位置：

- projects/mmdet3d_plugin/datasets/nuscenes_det_mapv2_dataset.py

3.4 第四步：在地图里生成四类向量 GT

`VectorizedLocalMapV2.gen_vectorized_samples(...)` 会依次生成：

- divider
- ped_crossing
- boundary
- centerline

最后把它们统一打包成：

- `gt_vecs_label`
- `gt_vecs_pts_loc`

随后在 dataset 里写成：

- `gt_map_vecs_label`
- `gt_map_vecs_pts_loc`

---

4. 位姿信息的具体作用

位姿信息不是“直接变成标签”，而是用于**选地图、裁 patch、做时序对齐**。

4.1 位姿公式要怎么理解

- 采用的列向量约定：

```
lidar2global = ego2global @ lidar2ego
```

也就是：

```
p_global = ego2global @ lidar2ego @ p_lidar
```

这与代码一致：

- tools/data_converter/nuscenes_converter.py
- projects/mmdet3d_plugin/datasets/nuscenes_det_occ_map_dataset.py

4.2 位姿如何用于选地图

在 `_add_vectormap_gt(...)` 里：

1. 先决定当前 sample 属于哪张地图。
   - 优先用 `map_location`
   - 若缺失，则用 `scene_name` 反查
2. 用 `map_location` 在 `self.vector_map.map_explorer` 中找到对应地图实例：

```
self.vector_map.map_explorer[location].map_api
```

3. 用位姿构造 patch：

```
map_pose = lidar2global_translation[:2]
rotation = Quaternion(lidar2global_rotation)
patch_box = (map_pose[0], map_pose[1], self.patch_size[0],
self.patch_size[1])
patch_angle = quaternion_yaw(rotation) / np.pi * 180
```

4. 在这个 patch 里取地图几何，生成 vector GT。

4.3 位姿如何用于历史帧对齐

在 detector 前向里：

- projects/mmdet3d_plugin/bevformer/detectors/bevformer.py

训练时：

```
len_queue = img.size(1)
prev_img = img[:, :-1, ...]
img = img[:, -1, ...]
prev_img_metas = copy.deepcopy(img_metas)

if self.keep_bev_history:
    prev_bev = self.obtain_all_history_bev(prev_img, prev_img_metas)
else:
    prev_bev = self.obtain_history_bev(prev_img, prev_img_metas)
```

测试时：

```
if self.can_bus_in_dataset:
    tmp_pos = copy.deepcopy(img_metas[0][0]['can_bus'][:3])
    tmp_angle = copy.deepcopy(img_metas[0][0]['can_bus'][-1])
    if self.prev_frame_info['prev_bev'] is not None:
        img_metas[0][0]['can_bus'][:3] -=
self.prev_frame_info['prev_pos']
```

PROF

```
        img_metas[0][0]['can_bus'][-1] -=
self.prev_frame_info['prev_angle']
```

作用是：

- 使用 `can_bus` 和历史保存的位姿
- 计算当前帧与上一帧的相对运动
- 把历史 BEV 对齐到当前帧坐标系

位姿信息主要用于两类任务：

- 地图 GT 生成：定位当前样本在地图中的位置
- 历史 BEV 对齐：把上一帧 BEV 变换到当前帧

---

5. 地图 JSON 的实际组织方式

当前这套代码依赖的是 nuScenes map expansion 风格的 JSON。

这类 JSON 不是单一表，而是由点、线、面、语义对象和拓扑关系共同组成的层级结构。

以 data/nuscenes/maps/expansion/boston-seaport.json 为例，顶层会包含：

- `version`
- `polygon`
- `line`
- `node`
- `road_segment`
- `road_block`
- `lane`
- `ped_crossing`
- `walkway`
- `stop_line`
- `carpark_area`
- `road_divider`
- `lane_divider`
- `traffic_light`
- `canvas_edge`
- `connectivity`
- `arcline_path_3`
- `lane_connector`

实际字段可以从样例中直接看到。例如：

- `token`、`polygon_token`、`node_tokens`、`line_token` 本质上都是 UUID （python import uuid;uuid.uuid4()）风格的唯一标识符，用于在不同表之间建立引用关系。它们由 nuScenes 地图导出或标注流程生成。
- `road_segment` 记录包含自身 `token`，并通过 `polygon_token` 引用对应面对象；例如：

PROF

    - `token = 00683936-1a08-4861-9ce5-bb4fc753dada`
    - `polygon_token = bea6cf31-59e5-48d9-8ca1-28312b5313d1`
- `connectivity` 记录以对象 `token` 作为 key，并显式保存 `incoming` / `outgoing`；例如：
    - `d190c816-c71b-4db2-9913-5f58d0b2c72d`
    - `incoming = [5c4ddfe1-21d3-4e91-bc85-23b8a4e6f855]`
    - `outgoing = [5e13747e-4ea8-422f-b286-ff3cd0a0f941, 8b7f8488-703b-4b0d-8de9-871bc0393ea7, f83b33f4-f455-4801-b38e-fded988784c3]`

5.1 点、线、面三层

- `node`
    - 最底层点，包含 `x/y`，带 `token`
- `line`
    - 一串 `node_tokens` 组成的线
- `polygon`
    - 一串 `exterior_node_tokens` 组成的面，可能还有 `holes`

5.2 语义对象层

- `lane`
    - 引用 `polygon_token`
    - 还带 `from_edge_line_token` / `to_edge_line_token`
- `lane_connector`
    - 引用 `polygon_token`
- `road_divider` / `lane_divider`
    - 引用 `line_token`
- `ped_crossing`
    - 引用 `polygon_token`
- `road_segment`
    - 引用 `polygon_token`

5.3 拓扑层

PROF

- `connectivity`
    - 为每个对象 token 记录 `incoming` / `outgoing`
- `arcline_path_3`
    - 提供中心线的参数化路径，供离散化和采样使用

这些 token / 拓扑结构的作用，就是支持：

- `map_api.extract_polygon(...)`
- `map_api.discretize_lanes(...)`
- `map_api.get_incoming_lane_ids(...)`
- `map_api.get_outgoing_lane_ids(...)`

---

6. 中心线拓扑是怎么做出来的

V2 版本和普通 MapTR 之间最大的差异之一，就是 `centerline` 类。

6.1 centerline 的语义来源

当前 V2 实现把 `centerline` 定义为：

- `lane_connector`
- `lane`

也就是这两类的中心路径。相关代码在：

- projects/mmdet3d_plugin/datasets/nuscenes_det_mapv2_dataset.py

6.2 单条中心线是怎么拿到的

在 `_get_centerline(...)` 里：

1. 找到当前 patch：
    - `patch = self.map_explorer[location].get_patch_coord(patch_box, patch_angle)`
2. 从 map API 中取对应 layer 的 records：
    - `records = getattr(map_api, layer_name)`
3. 取 `polygon_token` 并裁剪到 patch：
    - `map_api.extract_polygon(polygon_token)`
4. 调用：
    - `map_api.discretize_lanes([record['token']], 0.5)`
5. 变成 `LineString`，再裁剪到 patch
6. 做坐标变换

6.3 拓扑如何把多段中心线串起来

这一段是核心：

- 每条中心线记录都会带：
    - `incoming_tokens`
    - `outgoing_tokens`
- `union_centerline(...)` 会：
    - 把每条线自身的点顺序加入有向图 `nx.DiGraph()`
    - 再把前驱/后继 lane 的端点也连进来
    - 最终从 root 到 leaf 搜索完整路径

这样做的结果是：

- 不只是“局部 lane 片段”
- 而是“按拓扑拼接后的完整中心路径”

拓扑关系在 GT 构造中的体现如下：

- 拓扑不是额外输出一个 graph tensor
- 而是直接影响 GT 折线的构造方式

PROF

6.4 最终中心线 GT 的类别

`centerline` 最终被写成类别 3：

```
CLASS2LABEL = {
    'centerline': 3,
}
```

因此，训练 batch 中的 `gt_map_vecs_label` 会包含 `3`，对应 `centerline` 类别。

---

7. 训练时最终写进 batch 的 GT 长什么样

经过数据集和 pipeline，最终会写进训练 batch 的字段主要是：

- `gt_bboxes_3d`
- `gt_labels_3d`
- `gt_map_vecs_label`
- `gt_map_vecs_pts_loc`
- `img`
- `img_metas`

7.1 `gt_map_vecs_label`

这是每条向量对应的类别 id。

当前 V2 的类别编号是：

- 0：divider
- 1：ped_crossing
- 2：boundary
- 3：centerline

PROF

7.2 `gt_map_vecs_pts_loc`

这是每条向量的点序列。

当前约定是：

- 每条 polyline 固定采样 `map_num_pts = 20` 个点
- 因此其形状通常为：
  - `N × 20 × 2`

7.3 训练中的实际使用方式

在 head 的 loss 里：

- 主分支做 one-to-one matching
- 辅分支做 one-to-many 监督

- 辅助分割头还会用 `gt_map_vecs_pts_loc` 构建 BEV / PV segmentation target

相关代码：

- projects/mmdet3d_plugin/maptrv2/dense_heads/bevformer_det_map_head_apollo_v2.py

---

8. 常见问题

8.1 为什么一定要 `map_location`？

因为 nuScenes 有多张地图，`boston-seaport`、`singapore-onenorth` 等不是统一一张图。没有 `map_location`，数据集不知道该加载哪一个 `maps/expansion/<location>.json`。

8.2 为什么 `scene_name` 还能用？

因为部分 legacy infos 里可能没有 `map_location`，代码里会用 `scene_name` 反查 `map_location`，作为兜底。

8.3 为什么要用 `ego2global @ lidar2ego`？

因为当前实现采用列向量约定：先 lidar 到 ego，再 ego 到 global。顺序不能反。

8.4 为什么 centerline 要用拓扑拼接？

因为单个 lane 片段不足以表达道路连续结构。centerline 的目标是把 lane / lane_connector 的连接关系变成更完整的中心路径。

8.5 这个流程里有没有离线 map GT pkl？

当前这条链路没有必须的离线 map GT pkl。主要是：

- 离线生成 infos
- 在线生成 vector GT

PROF

附录：最关键的文件路径

- 配置：
  - projects/configs/bevformer/bev_tiny_det_mapv2.py
- 数据集：
  - projects/mmdet3d_plugin/datasets/nuscenes_det_mapv2_dataset.py
  - projects/mmdet3d_plugin/datasets/nuscenes_det_occ_map_dataset.py
- 检测器：
  - projects/mmdet3d_plugin/bevformer/detectors/bevformer.py
- 离线转换：
  - tools/create_data.py
  - tools/data_converter/nuscenes_converter.py
- 可视化：
  - tools/simulate_and_vis_map_gt.py

- tools/debug_map_gt_one_sample.py